

Four tasks (T1...T4) are using three resources (R1...R3). Mutual exclusion is implemented using semaphores. Your task is to detect deadlocks while the tasks are requesting resources.

The tasks' programs are executed step-by-step. In each step every task executes a single instruction (if it is not blocked). Tasks allocate (if it is free) and free a single resource in every step in T1...T4 order.

Resource allocations by steps:

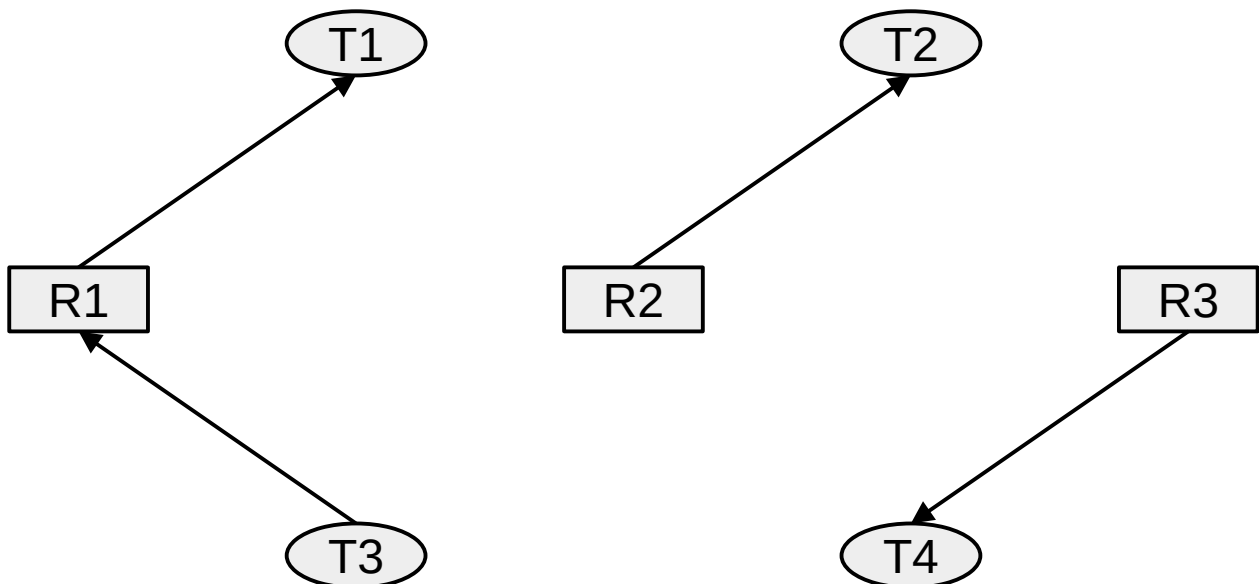
T1:	+R1,	+R3,	-R1,	+R2,
T2:	+R2,	-R2,	+R2,	-R2,
T3:	+R1,	+R2,	+R3,	-R3,
T4:	+R3,	-R3,	+R1,	+R2,

When a resource is freed the first task waiting for it gets the resource immediately. It may execute its next instruction in the same step if the task that released the resource has a lower task ID.

Draw the resource allocation graphs after each step finished and write down the instructions executed.

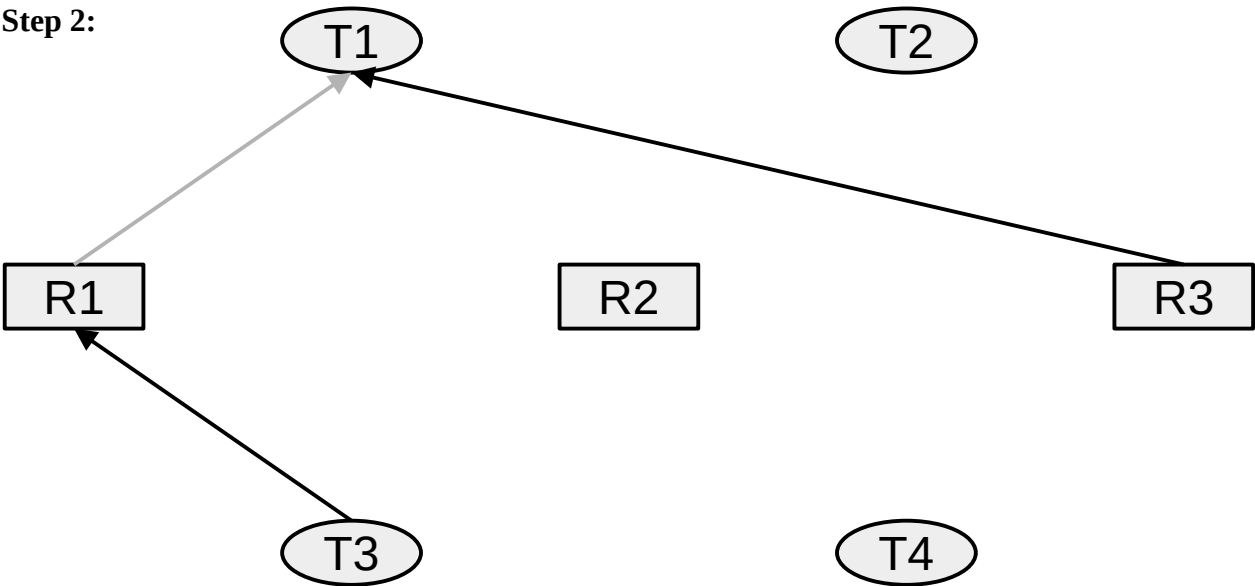
(gray edges denote those, that were already in the graph before the given step)

Step 1:



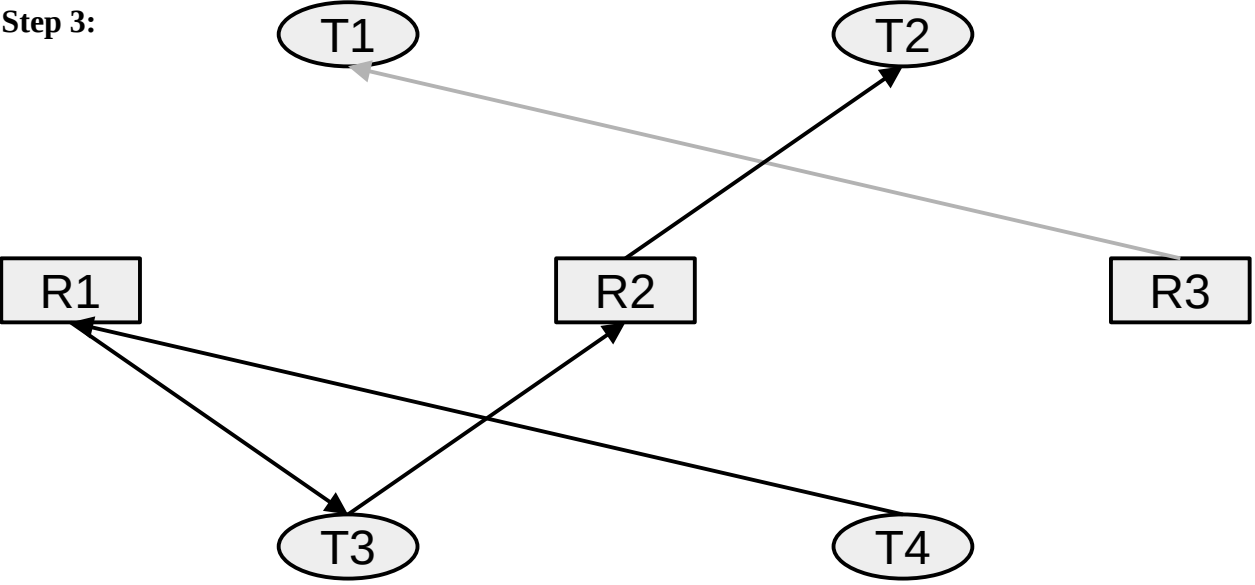
T1:	+R1,	+R3,	-R1,	+R2,
T2:	+R2,	-R2,	+R2,	-R2,
T3:	+R1,	+R2,	+R3,	-R3,
T4:	+R3,	-R3,	+R1,	+R2,

Step 2:



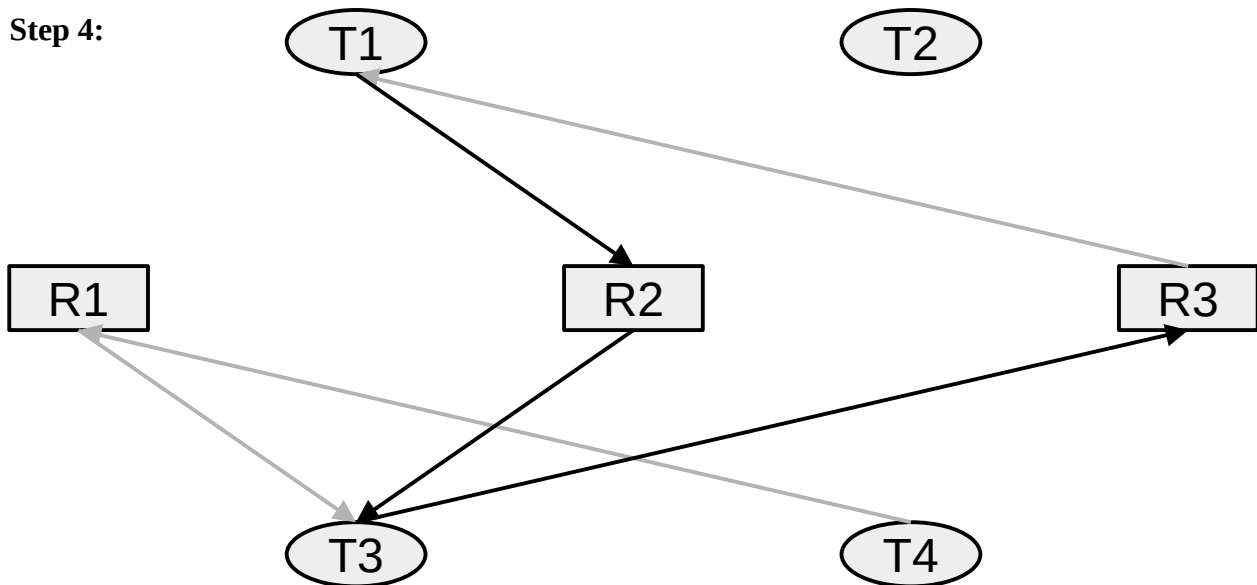
T1:	+R1,	+R3,	-R1,	+R2,
T2:	+R2,	-R2,	+R2,	-R2,
T3:	+R1,	+R2,	+R3,	-R3,
T4:	+R3,	-R3,	+R1,	+R2,

Step 3:



T1:	+R1,	+R3,	-R1,	+R2,
T2:	+R2,	-R2,	+R2,	-R2,
T3:	+R1,	+R2,	+R3,	-R3,
T4:	+R3,	-R3,	+R1,	+R2,

Step 4:



T1:	+R1,	+R3,	-R1,	+R2,
T2:	+R2,	-R2,	+R2,	-R2,
T3:	+R1,	+R2,	+R3,	-R3,
T4:	+R3,	-R3,	+R1,	+R2,

Is there a deadlock in the system? (How can you detect it?)

Yes, there is.

We can see that there is the directed cycle $T1 \rightarrow R2 \rightarrow T3 \rightarrow R3 \rightarrow T1$, i.e. T1 and T2 are both waiting for each other, and thus cannot proceed any further.

What are the four necessary conditions for deadlocks?

...

What can we do after the deadlock occurs?

...